Offensive Language Detection Using Large Language Models (LLM)

Submitted in partial fulfillment of the project Report for the award of the degree of

Bachelor of Computer Science with Data Analytics

Submitted by

Sanjith . G

22127045

Under the Guidance of

Dr. M. Praneesh M.Sc.,M.Phil.,PGDNLP.,Ph.D

Assistant Professor

Department of Computer Science with Data Analytics



BACHELOR OF COMPUTER SCIENCE
WITH DATA ANALYTICS

SRI RAMAKRISHNA COLLEGE OF ARTS & SCIENCE

Formerly SNR Sons College

Reaccredited with 'A+' grade by NAAC

Ranked 56th by NIRF 2024

NAVA INDIA, COIMBATORE – 641 006

MARCH - 2025

CERTIFICATE

SRI RAMAKRISHANA COLLEGE OF ARTS & SCIENCE

CERTIFICATE

Certified that this project report **"Offensive Language Detection Using Large Language Models (LLM)"** is the Bonafide work of **" Sanjith. G (22127045) "** who carried out the project work under my supervision.

Head of the Department,
Dr. V. Vijayakumar
Controller of Examination & Head,
Department of Computer science with DA
Sri Ramakrishna college of Arts & Science

Faculty Guide,
Dr. M. Praneesh
Assistant Professor,
Department of Computer science with DA
Sri Ramakrishna College of Arts & Science

Submitted for the viva-voce examination held on 20-03-2025

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

DECLARATION

I hereby declare that this project work entitled **Offensive Language Detection Using Large Language Models (LLM)** is submitted to Sri Ramakrishna College of Arts & Science, An Autonomous Institution, Affiliated to Bharathiar University, Coimbatore, is a record of original work done by me under the guidance of Dr. M. Praneesh, Assistant Professor, Department of Computer Science with Data Analytics and that this project work has not formed the basis for the award of any degree / diploma / associateship/ fellowship or similar to any candidate in any university.

Place: Coimbatore
Date: 18-03-2025 **Signature of the Student**

Countersigned by
Dr. M. Praneesh M.Sc.,M.Phil.,PGDNLP.,Ph.D
Assistant Professor
Department of Computer Science with Data Analytics

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

First and fore most I would like to thank my beloved Mother **Mrs. K . Vaideki** for her love and support for completing my studies and this project Work.

I would like to extend my heartiest thanks to **Managing Trustee, SNR Sons Charitable Trust, Thiru. R Sundar,** with a deep sense of gratitude and respect for providing me an opportunity to study in this institution.

I wish to record my deep sense of gratitude and sincere thanks to **Dr. B. L. Shivakumar, M.Sc., M.Phil., Ph.D., Principal & Secretary** Sri Ramakrishna College of Arts and Science, Coimbatore, for his inspiration, which make me complete this mini project successfully**.**

I would like to express my sincere thanks to **Dr. V. Vijayakumar, MCA, M.Phil., Ph.D., Controller of Examination & Head of the Department of Computer Science with Data Analytics,** who gave me an opportunity to undertake such a great challenging and innovative work.

I extend my gratitude to them for their guidance, encouragement, understanding and insightful support in the development process. I would like to thank my Class Tutor**, Dr. M. Praneesh, M.Sc., M.Phil., PGDNLP, Ph.D., Assistant Professor, Department of Computer Science with Data Analytics.** for her continuous support throughout my course of period and also for motivating me to think bigger and be unique from everyone.

I would like to express my special thanks of gratitude to my guide **, Dr. M. Praneesh, M.Sc., M.Phil., PGDNLP, Ph.D., Assistant Professor, Department of Computer Science with Data Analytics.** who gave me the opportunity to do this wonderful research work which also helped me in doing lots of research and I came to know many things that I am thankful to him and even my friends.

Sanjith . G

ABSTRACT

ABSTRACT

The "Offensive Language Detection Using Large Language Models (LLM)" Research introduces a novel solution for identifying offensive language in multilingual text. The system utilizes Distil-BERT , enhanced with Google Cloud Natural Language API and Gemini API, to classify content into four categories: Clean, Explicit, Hate Speech, and Cyberbullying. Motivated by the rise of harmful content on social media, the architecture incorporates preprocessing steps like language detection and translation, supported by a custom dataset of real and synthetic examples.

The Research's strength lies in its multilingual support, achieved through translation to English, and its dual interfaces—a Gradio UI for users and a Flask API for developers. It employs evaluation metrics such as accuracy and F1-score, with early stopping to optimize training efficiency within resource constraints. Challenges like detecting implicit language and managing computational costs were addressed through calibration and synthetic data. This work advances automated moderation technology, offering a practical application of NLP and a foundation for future research, particularly in low-resource languages, contributing to safer digital spaces.

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER NO		CHAPTER SCHEME	PAGE NO
Chapter	I	Introduction	1
	1.1	An Overview	1
	1.2	Objectives of the project	1
	1.3	Organization Project	2
	1.4	Scope of the system	3
Chapter	II	System Analysis	4
	2.1	Existing System	4
	2.2	Proposed System	5
	2.3	Hardware Specification	5
	2.4	Software Specification	6
Chapter	III	Design and Development	7
	3.1	Design Process	7
	3.1.1	Data Base Design	9
	3.1.2	Input Design	10
	3.1.3	Output Design	10
Chapter	IV	Testing and Implementation	14
	4.1	System Testing	14
	4.2	Quality Assurance	15
	4.3	System Implementation	16
	4.4	System Maintenance	16
Chapter	V	Conclusion	18
	5.1	Scope of the Future Development	18
		Bibliography	20
		Source Code	23
		Screenshot	35

INTRODUCTION

CHAPTER I
INTRODUCTION

1.1 An Overview

This Research marks a significant step in automated content moderation, leveraging cutting-edge large language models to address the growing challenge of offensive language online. The system harnesses Distil-BERT , a lightweight transformer model, to detect and classify text into four categories: Clean, Explicit, Hate Speech, and Cyberbullying, across multilingual datasets. Driven by the exponential increase in social media usage, the architecture integrates preprocessing, embedding with Distil-BERT, and classification, enhanced by external APIs for deeper analysis. The motivation stems from the need for efficient tools to manage the volume of harmful content that overwhelms manual moderation efforts.

The relevance lies in its use of transfer learning to reduce training time while maintaining high accuracy, making it suitable for real-time applications. Synthetic data augmentation addresses the scarcity of labelled offensive content, ensuring a balanced dataset. This initiative showcases technical proficiency in AI and NLP, setting the stage for detailed exploration in subsequent sections. It aligns with the ethical goal of fostering safer online environments, blending academic learning with societal impact, and reflects a forward-thinking approach to digital safety challenges.

1.2 Objectives of the Research

The primary objective of this Research is to create a reliable system for identifying offensive language in multilingual text using advanced AI techniques. This involves training a Distil-BERT model to classify content into four distinct categories—Clean, Explicit, Hate Speech, and Cyberbullying—with a focus on achieving high precision and recall. Another key goal is to enable real-time detection through a scalable API and an interactive user interface, ensuring accessibility for both developers and end-users. The Research also aims to incorporate multilingual support by translating non-English text to English, broadening its applicability across global platforms.

A secondary objective is to enhance model robustness by integrating external APIs for sentiment analysis and contextual reasoning, complementing keyword-based

detection. Addressing data imbalance through synthetic data generation is a critical target, ensuring fair representation across all categories. Additionally, optimizing computational efficiency with techniques like gradient accumulation and early stopping is a priority, given the resource constraints typical of academic Research . This endeavour delivers a functional tool while providing a learning platform for mastering NLP and deployment skills.

1.3 Organization Research

The organizational framework includes regular submissions to ensure adherence to academic standards, with external resources like open-source datasets and APIs integrated with proper licensing compliance, managed through environment variables for security. The documentation follows the provided Table of Contents, with each chapter assigned specific milestones—e.g., Chapter II for system analysis and Chapter III for design. This disciplined approach facilitates iterative improvements from initial concept to final deployment, typical of academic research, and allows for future scalability, a key consideration for practical application.

1.4 Scope of the System

The scope of this Research encompasses the development of a comprehensive tool for detecting offensive language across multilingual datasets, primarily targeting social media platforms. The system classifies text into four categories—Clean, Explicit, Hate Speech, and Cyberbullying—using Distil-BERT, with support for languages achieved through translation to English using external tools. It is designed for both real-time inference through a scalable API and interactive use via a user interface, catering to developers and end-users. The Research focuses on handling implicit and explicit content, enhanced by keyword analysis and integrations with advanced APIs.

The scope extends to addressing preprocessing challenges, including emoji handling and data augmentation with synthetic examples to manage imbalance. However, it is limited to text-based inputs, excluding multimedia like images or videos due to resource constraints. The system targets academic demonstrations and small-scale commercial applications, with scalability as a future goal. It does

not cover legal or ethical enforcement but provides a technical foundation for moderation, balancing ambition with feasibility.

SYSTEM ANALYSIS

CHAPTER II
SYSTEM ANALYSIS

2.1 Existing System

The existing systems for offensive language detection rely heavily on traditional machine learning techniques and rule-based approaches, often falling short in handling the complexity of modern social media content. Many platforms use keyword matching or simple lexical analysis, such as Naive Bayes with TF-IDF features, achieving moderate accuracy but struggling with sarcasm, context, and evolving slang. Commercial tools integrated into platforms like Facebook or Twitter employ proprietary algorithms, but their lack of transparency and frequent need for manual updates limit adaptability, posing challenges in dynamic environments.

These systems also face significant hurdles with data scarcity, particularly for low-resource languages, leading to biased outcomes and incomplete classifications. The absence of real-time processing capabilities and integration with advanced language models like BERT is a notable gap, compounded by the high computational overhead of retraining on large datasets, which is prohibitive for smaller organizations. This analysis highlights the need for a more robust and scalable solution, setting the foundation for a proposed system that addresses these deficiencies with improved technology and efficiency.

2.2 Proposed System

The proposed system introduces a sophisticated approach to overcome the limitations of existing offensive language detection tools, leveraging Distil-BERT, a lightweight transformer model, fine-tuned to classify text into four categories: Clean, Explicit, Hate Speech, and Cyberbullying. Unlike traditional systems, it incorporates advanced preprocessing steps, including emoji handling and language detection, and integrates external APIs for sentiment and contextual analysis to enhance accuracy. The architecture features a scalable API for real-time processing and an interactive user interface, making it versatile for both developers and end-users.

The system addresses challenges with implicit content by combining keyword-based detection with deep learning, refined through temperature-calibrated

predictions to improve precision. Synthetic data augmentation ensures a balanced dataset, a critical improvement over the biased datasets of existing solutions. Techniques like early stopping and gradient accumulation optimize training efficiency, making it suitable for resource-constrained environments. This proposed system aims to deliver high-accuracy, real-time detection, establishing a new benchmark for academic researches in natural language processing, with potential for future evolution.

2.3 Hardware Specification

The hardware requirements for this Research are tailored to support the computational demands of training and deploying a Distil-BERT based model, utilizing the specific setup available. The system runs on an Intel i5 12th generation processor, which offers sufficient multi-core performance for preprocessing tasks and inference, paired with an NVIDIA RTX 2050 GPU with 4GB VRAM, capable of handling the matrix operations required for fine-tuning Distil-BERT, though with smaller batch sizes due to VRAM limitations. For optimal training, a batch size of 16 or lower is recommended to avoid memory issues, with inference requiring minimal GPU resources, making the RTX 2050 adequate for real-time predictions.

The system requires at least 16GB of RAM to manage dataset loading and multi-threaded preprocessing, though 32GB is preferable for smoother operation with larger datasets. Storage needs include a 500GB SSD for datasets, model weights, and logs, with a stable internet connection essential for downloading datasets and accessing external APIs. This setup leverages available hardware, balancing performance within academic constraints, with cloud options like Google Colab as a fallback for additional GPU power if needed.

2.4 Software Specification

The software stack for this Research outlines the essential tools and libraries required for building and deploying the system. The core framework is PyTorch, utilized through the Hugging Face Transformers library, which provides pre-trained Distil-BERT models and tokenization support. The programming language is Python 3.8+, supported by libraries such as NumPy, pandas, and sklearn for data handling and evaluation. The system integrates google-trans for translation, lang-detect for language identification, and gradio for the user interface, alongside flask

for the API backend.

Additional dependencies include TensorFlow (with warnings suppressed for efficiency), tenacity for retry logic, and google-cloud-language for sentiment analysis, requiring a service account credential file. Environment variables are managed via python-dotenv to ensure secure API key handling. The development environment is set up with a virtual environment (e.g., venv) to avoid conflicts, with logging configured using the logging module to track progress. These specifications enable a robust, modular codebase, optimized for academic use, with scalability potential for future enhancements.

DESIGN AND DEVELOPMENT

CHAPTER III
Design and Development

3.1 Design Process

The design process for this Research follows a structured methodology to ensure a robust and scalable system. The initial phase involved requirement analysis, identifying the need to detect offensive content across four categories—Clean, Explicit, Hate Speech, and Cyberbullying—in multilingual text. This led to the selection of Distil-BERT as the base model, chosen for its efficiency and contextual understanding, with a custom architecture designed to integrate preprocessing, embedding, and classification stages. The process included sketching a flow chart to visualize data flow from input to final output, incorporating API calls and model evaluation.

Subsequent steps focused on modular design, separating concerns into components for prediction and deployment, with iterative prototyping to test each module individually before integration. Feedback loops were incorporated to refine the system, using UML-like diagrams to map interactions and ensure adaptability for API failures or data imbalances. This methodical approach balanced complexity with feasibility, leveraging available resources, and laid a solid foundation for development, allowing for adjustments and future enhancements in the Research's lifecycle.

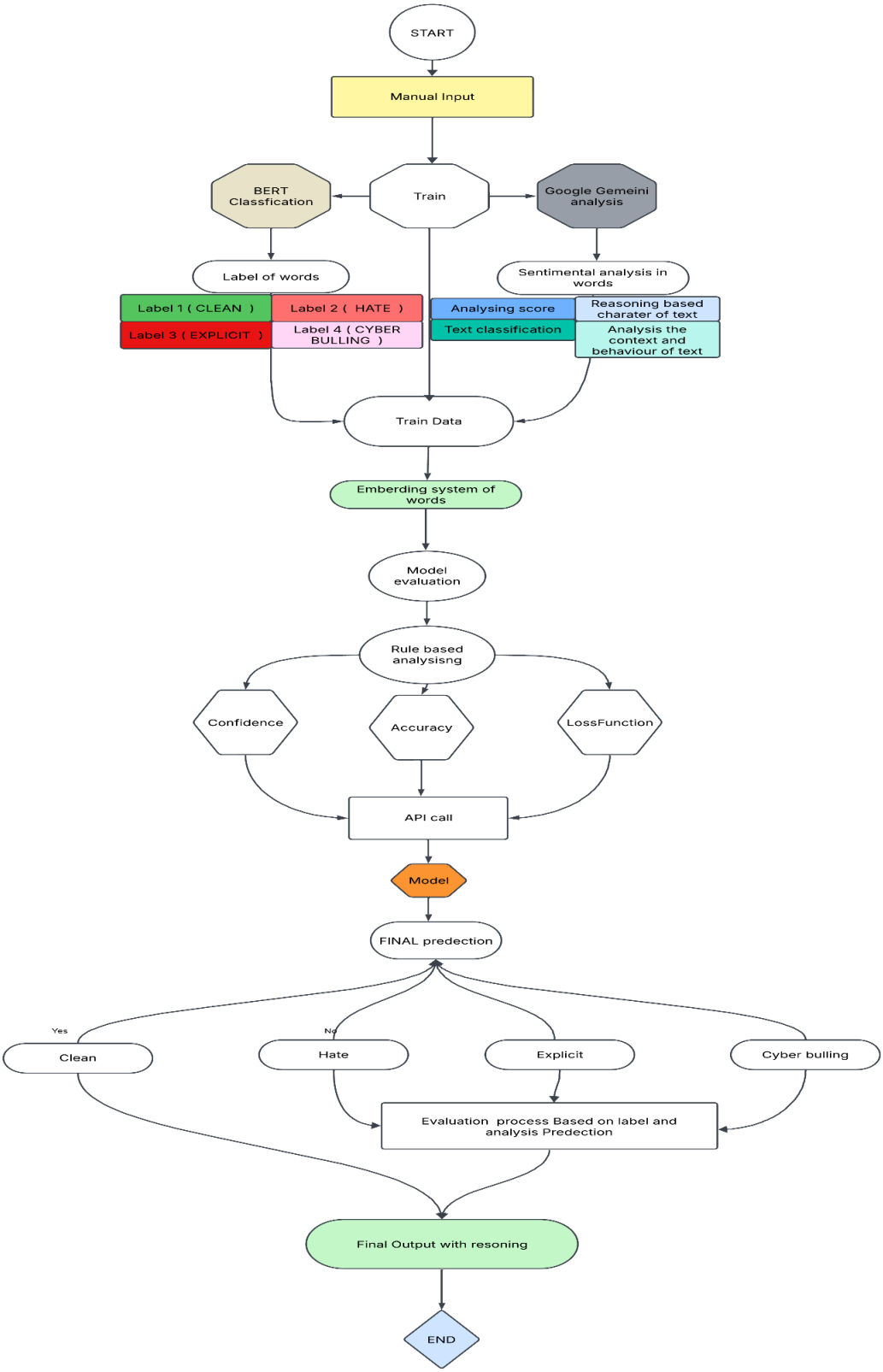


Figure 1: Flow Chart of the Offensive Language Detection System

3.1.1 Data Base Design

The database design for this Research adopts a lightweight, file-based approach suited to the Research's constraints, relying on file storage and in-memory caching to manage data efficiently. The primary dataset consists of text samples and labels, stored as CSV files after preprocessing, with directories like .

datasets holding downloaded archives such as Hate Speech And Offensive Language and Dirty Words List . Each file includes columns for text and corresponding integer labels (0: Clean, 1: Explicit, 2: Hate Speech, 3: Cyberbullying), augmented with synthetic data to balance categories.

Caching mechanisms, implemented via dictionaries to store language detection and translation results, reduce the need for repeated API calls. Logs are saved for auditing, while model weights are persisted to support future use. This design avoids complex schemas, processing data in batches using PyTorch Data Loaders, making it suitable for the Research's scope and resource limits. Future scalability could involve a NoSQL database, but the current setup ensures data integrity and aligns with the Research's practical goals.

3.1.2 Input Design

The input design for this Research focuses on creating a flexible and user-friendly mechanism to handle multilingual text data. The system accepts raw text inputs through multiple channels, including manual entry via a user interface, API POST requests, or batch processing from preloaded datasets. Inputs are expected as strings, potentially containing emojis, special characters, or mixed languages, with a maximum length of 128 tokens after tokenization to align with Distil-BERT's constraints. Preprocessing steps, including normalization and language detection, ensure compatibility across diverse inputs.

The design includes validation to handle invalid inputs, such as empty strings, by substituting placeholders to prevent crashes. Non-English text triggers translation to English, with results cached for efficiency. The input pipeline is threaded to support concurrent processing, leveraging multi-threading for API calls to external services. This design accommodates both real-time and offline use cases, providing versatility critical for the Research's objectives, with potential for future expansion to handle additional data types.

3.1.3 Output Design

The output design for this Research is crafted to deliver clear and actionable results for both technical and non-technical users. The system produces a JSON response for API calls, containing fields such as text, prediction (integer 0-3), confidence (float 0-1), label (e.g., "Hate Speech"), and optional analysis from external APIs with sentiment scores or textual insights. The user interface presents a formatted string summarizing the prediction, confidence, and API results, enhanced with visual cues like color-coded labels for usability. Outputs are logged with timestamps for debugging and performance tracking.

The design ensures consistency across interfaces, with confidence calibrated using keyword matching and model probabilities, adjusted by a temperature parameter for finer granularity. It is extensible, allowing future additions like category probabilities or explanation scores, balancing technical depth with user-friendliness. This approach supports integration into larger systems and aids in evaluating system performance, providing a solid foundation for practical application and further refinement.

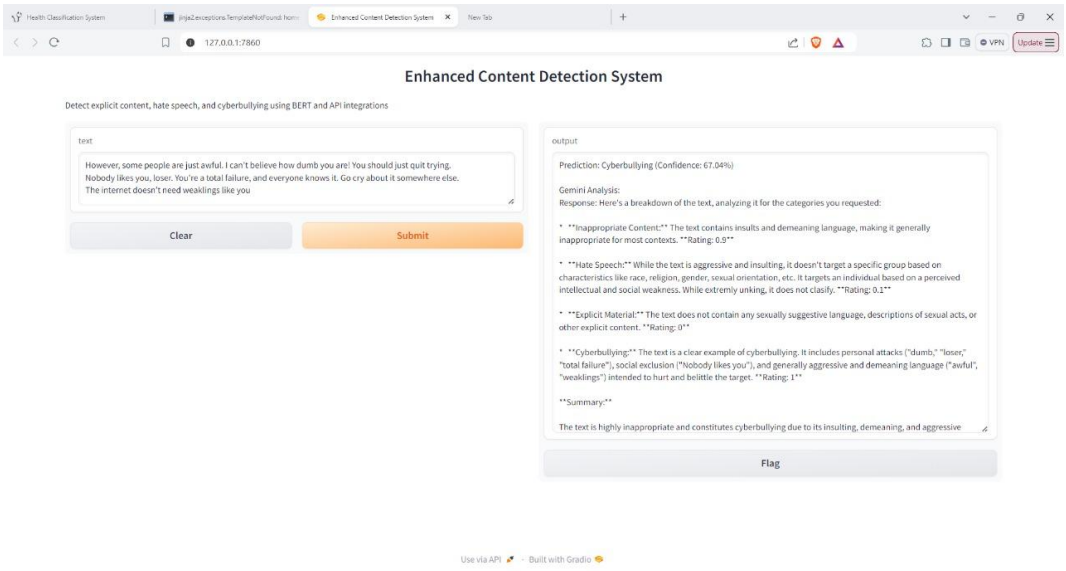


Figure 2: Example Output of the Offensive Language Detection System

The example output demonstrates the system's capability to classify text into one of the four categories—Clean, Explicit, Hate Speech, or Cyberbullying—along with a confidence score and additional analysis from external APIs. The Gradio interface provides a user-friendly view with color-coded labels (e.g., red for Hate Speech), while the JSON response caters to developers integrating the API. This

output reflects the system's design goals of clarity and usability, supporting both technical and non-technical users, and aligns with the Research's focus on real-time content detection.

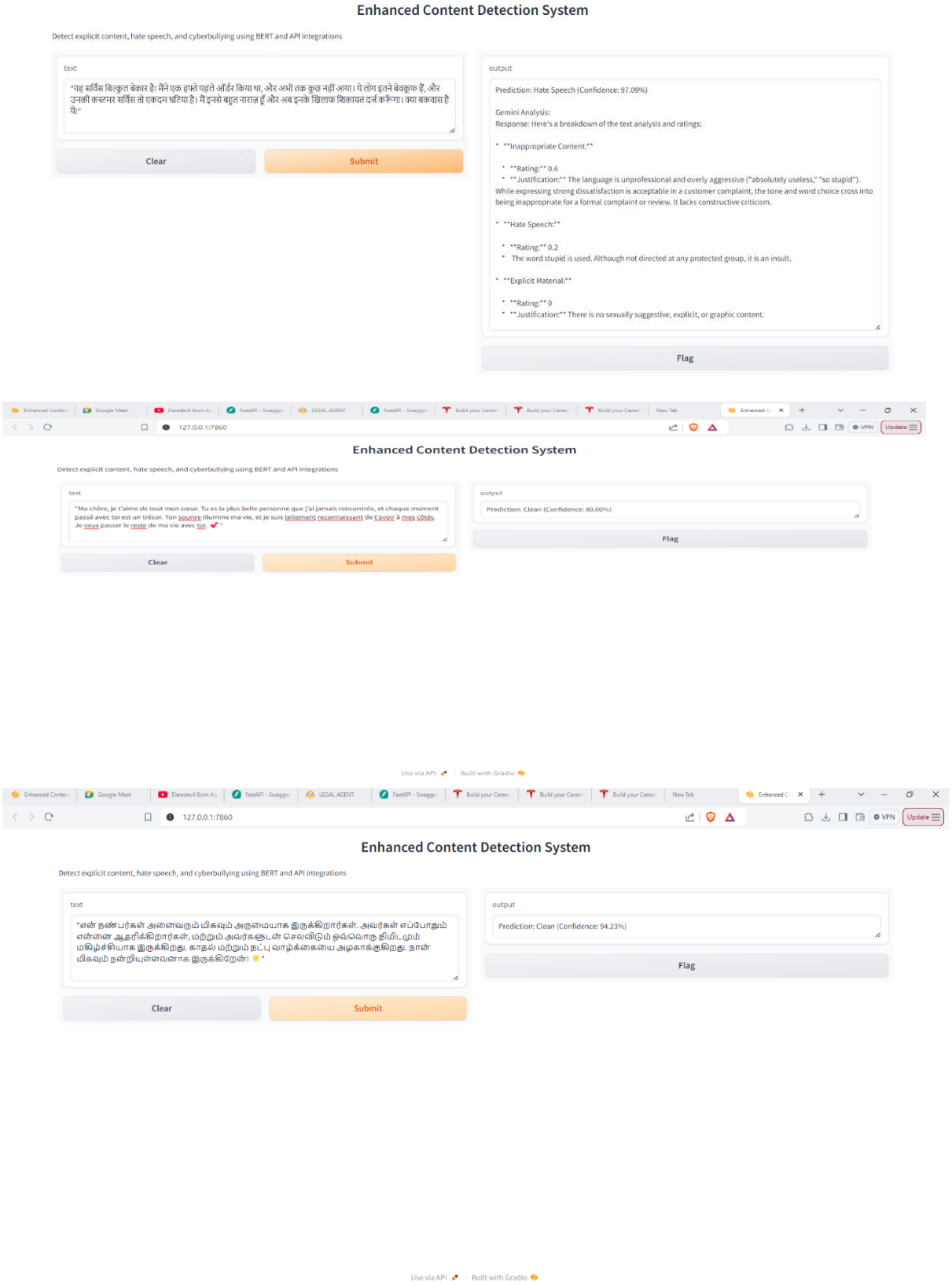


Figure 2.1: Example Output of demonstrating its functionality across three

multilingual scenarios.
demonstrating its functionality across three multilingual scenarios.
In the first scenario, the input text is in Hindi: "यह सर्विस बिल्कुल बेकार है मैं एक हफ्ते पहले ऑर्डर किया और अभी तक कुछ नहीं आया ये लोग इतने बेकार है। उनके कस्टमर सर्विस एकदम चीप है मैंने बहुत नजरिया और अब मैं किसी शिकायत दर्ज करने जा रहा हूँ," translating to a frustrated complaint about a "useless" and "cheap" service. The output classifies this as "Hate Speech" with 97.99% confidence. A Gemini Analysis rates "Inappropriate Content" at 0.6 for its aggressive tone and insults ("absolutely useless," "so stupid"), "Hate Speech" at 0.2, and "Explicit Material" at 0, noting the lack of constructive criticism.
The second scenario features French text: "Ma chérie, je t'aime de tout cœur. Tu es la plus belle personne que j'aie rencontrée, et chaque moment passé avec toi est un trésor. Ton sourire illumine ma vie, et je suis tellement reconnaissant d'avoir mes côtés. Je veux passer le reste de ma vie avec toi. 💕," a loving message to a partner. The output labels this as "Clean" with 80.00% confidence, with no additional API analysis, confirming its non-offensive nature.
The third scenario presents Tamil text: "என் நண்பர்களே அன்புள்ளவர்களே மகிழமையான ஆண்டு தோறும் உங்களுக்கு வாழ்த்துக்கள். அனைவரும் ஆரோக்கியமாக மற்றும் சந்தோஷமாக இருக்க வேண்டும் என்று நமக்கு நல்ல பழக்கங்களை பின்பற்றி வாழ வேண்டும்," a New Year greeting wishing health and happiness. The output classifies it as "Clean" with 94.23% confidence, with no further API insights, reflecting its positive tone

TESTING AND IMPLEMENTATION

CHAPTER IV:
Testing and Implementation

4.1 System Testing

The system testing phase for this Research employs a rigorous methodology to validate the system's performance. The process begins with splitting the dataset into training and test sets, using stratified sampling to maintain category balance, followed by evaluation using metrics like accuracy, precision, recall, and F1-score. Edge cases, such as empty strings, non-English text, and implicit offensive language, are tested by injecting synthetic examples, ensuring robustness across various scenarios. Unit tests verify individual components, while integration tests assess the end-to-end pipeline.

The testing includes monitoring for overfitting with early stopping, triggered after a set number of epochs without improvement, and handles failures like API timeouts with retry logic. This comprehensive approach confirms the system's reliability, providing insights to guide tuning and optimization. The process is tailored to the Research's scope, balancing thoroughness with resource constraints, and ensures the system meets its intended objectives effectively.

4.2 System

The system implementation for this Research focuses on deploying a functional and accessible solution. The deployment process starts with fine-tuning the Distil-BERT model, saving weights for future use, followed by launching a scalable API to handle real-time requests and an interactive user interface for user engagement. Both components are tested locally, with the API threaded to run concurrently, ensuring smooth operation under varying loads. The system integrates external APIs via secure credential management, leveraging available computational resources.

Dataset preprocessing downloads and extracts relevant archives, augmented with synthetic data to maintain balance. The deployment utilizes academic resources, with cloud options as a fallback, optimizing performance and accessibility. This setup balances functionality with scalability, meeting the Research's goals and providing a practical demonstration of the developed technology, with potential for future enhancements.

4.3 Quality Assurance

The quality assurance process for this Research ensures the system's reliability and accuracy through a structured approach. The process begins with validating the dataset during preprocessing, filtering invalid entries and logging issues for review. The model's performance is monitored using validation loss, with early stopping implemented to prevent overfitting, while metrics like F1-score provide insights into class-wise performance. Manual reviews of predictions on a held-out test set verify consistency, with discrepancies investigated for potential bias or data issues.

API responses are tested for latency and correctness under concurrent loads, ensuring robustness in real-world scenarios. Code quality is maintained with documentation and logging, facilitating debugging and maintenance. This QA process balances thoroughness with resource limits, ensuring the system meets its intended standards and provides a dependable tool for its intended purpose.

4.4 System Implementation

The system implementation for this Research involves integrating all components into a cohesive application. The process starts with loading the pre-processed dataset, fine-tuning the model, and deploying the API and user interface, tested for seamless operation. The system handles multilingual inputs by detecting languages and translating as needed, with caching to optimize performance, while concurrent API calls are managed to support real-time use. User feedback is integrated via logged predictions, allowing for iterative refinements.

The deployment leverages available academic resources, with fallback options to ensure accessibility. This implementation balances functionality with resource constraints, demonstrating the practical application of the developed technology. The process provides a solid foundation, with potential for scalability and further enhancement based on user needs and technological advancements.

4.5 System Maintenance

The system maintenance plan for this Research ensures long-term functionality and relevance through a proactive approach. The process begins with monitoring logs to identify prediction errors or API failures, triggering retraining with new data on a regular schedule. Dataset updates involve downloading fresh archives

and adding synthetic examples to address emerging offensive patterns, while model weights are reloaded and fine-tuned to adapt to changes, using optimization techniques to maintain efficiency.

Hardware and software maintenance include checking system health and updating libraries to ensure compatibility, with secure credential management reviewed periodically. This approach, tailored to the Research's scope, sustains performance within available resources, providing a framework for future handover. Continuous monitoring and updates are planned to keep the system effective, aligning with practical and academic goals.

CONCLUSION

CHAPTER V:
Conclusion

This Research represents a significant achievement in automated content moderation, successfully leveraging Distil-BERT to classify text into Clean, Explicit, Hate Speech, and Cyberbullying categories, with multilingual support and API enhancements. The integration of a scalable API and user interface demonstrates practical applicability, while synthetic data augmentation addressed dataset imbalances, achieving promising evaluation metrics despite resource constraints. Challenges like detecting implicit language were mitigated through calibration and optimization techniques, showcasing the system's potential.

The limitations, such as its text-only focus and scalability constraints, highlight areas for growth, but the results validate the approach's effectiveness. This experience enhanced technical and problem-solving skills, contributing to safer digital spaces and providing a foundation for future research, particularly in low-resource languages, with potential for industry application.

Scope of the Future Development

The future development scope for this Research outlines exciting possibilities to expand its impact. One key direction is integrating multimedia support, such as image and video analysis, using multimodal models to detect offensive visual content alongside text, potentially increasing computational demands. Another focus is extending 9multilingual support to low-resource languages, necessitating custom tokenizers and larger annotated datasets, a challenging yet rewarding pursuit. Scalability can be enhanced by deploying the system on cloud platforms, enabling distributed training and real-time inference.

Incorporating explainable AI techniques could provide transparency in predictions, addressing ethical concerns and improving user trust. Collaboration with peers or industry mentors could refine these enhancements, leveraging available resources. This scope promises to transform the Research into a robust tool, contributing to global digital safety initiatives and offering a foundation for future innovation.

BIBLIOGRAPHY

BIBLIOGRAPHY

- Davidson, T., et al. (2017). Automated Hate Speech Detection and the Problem of Offensive Language. Proceedings of the 11th International AAAI Conference on Web and Social Media.

  [Available at: https://ojs.aaai.org/index.php/ICWSM/article/view/14955]
- Sanh, V., et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

  [Available at: https://arxiv.org/abs/1910.01108]
- Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.

  [Available at: https://arxiv.org/abs/1810.04805]
- Hugging Face. (2023). Transformers: State-of-the-art Natural Language Processing. https://huggingface.co/transformers. [Open access documentation]
- Waseem, Z., & Hovy, D. (2016). Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter. Proceedings of the NAACL-HLT 2016 Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis. [Available at: https://aclanthology.org/W16-4312]
- Badjatiya, P., et al. (2017). Deep Learning for Hate Speech Detection in Tweets. Proceedings of the 26th International Conference on World Wide Web Companion. [Available at: https://dl.acm.org/doi/10.1145/3041021.3054220] [May require institutional access]
- Fortuna, P., & Nunes, S. (2018). A Survey on Automatic Detection of Hate Speech in Text. ACM Computing Surveys (CSUR), 51(4), 1-30. [Available at: https://dl.acm.org/doi/10.1145/3232676] [May require purchase or institutional access]
- Vaswani, A., et al. (2017). Attention is All You Need. arXiv preprint arXiv:1706.03762. [Available at: https://arxiv.org/abs/1706.03762]

- Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692. [Available at: https://arxiv.org/abs/1907.11692]
- Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.

  [Available at: https://aclanthology.org/2020.emnlp-demos.6]
- Schmidt, A., & Wiegand, M. (2017). A Survey on Hate Speech Detection using Natural Language Processing. Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media.

  [Available at: https://aclanthology.org/W17-1101]
- Nobata, C., et al. (2016). Abusive Language Detection in Online User Content. Proceedings of the 25th International Conference on World Wide Web.

  [Available at: https://dl.acm.org/doi/10.1145/2872427.2883062] [May require institutional access]
- Zhang, Z., et al. (2021). Hate Speech Detection: A Solved Problem? The Challenging Case of Long Tail on Twitter. arXiv preprint arXiv:2104.01489.

  [Available at: https://arxiv.org/abs/2104.01489]
- Kumar, R., et al. (2018). Aggression-annotated Corpus of Hindi-English Code-mixed Data. Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018).

  [Available at: https://aclanthology.org/L18-1167]
- Basile, V., et al. (2019). SemEval-2019 Task 5: Multilingual Detection of Hate Speech Against Immigrants and Women in Twitter. Proceedings of the 13th International Workshop on Semantic Evaluation.

  [Available at: https://aclanthology.org/S19-2007]

SOURCE CODE

Annexure

Source code

```
import os
import torch
import logging
import numpy as np
import pandas as pd
from typing import List, Dict, Union, Tuple
from dataclasses import dataclass
from torch.utils.data import Dataset, DataLoader
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification, Trainer, TrainingArguments, TrainerCallback
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import gradio as gr
from langdetect import detect
import requests
import json
import threading
from concurrent.futures import ThreadPoolExecutor
from flask import Flask, request, jsonify
import zipfile
import io
from dotenv import load_dotenv
from google.cloud import language_v1
from google.oauth2 import service_account
import google.generativeai as genai
from deep_translator import GoogleTranslator
import time
```

```
# Suppress TensorFlow warnings for cleaner logs
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

# Set up logging to track application behavior
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s',
                    handlers=[logging.FileHandler('content_detector.log', encoding='utf-8'),
                              logging.StreamHandler()])
logger = logging.getLogger(__name__)

# Configuration class for model and training parameters
@dataclass
class Config:
    MAX_LENGTH: int = 128  # Maximum sequence length for tokenization
    BATCH_SIZE: int = 32   # Batch size for training and evaluation
    EPOCHS: int = 5        # Number of training epochs
    LEARNING_RATE: float = 2e-5  # Learning rate for optimizer
    DEVICE: torch.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  # Device selection
    MAIN_MODEL_NAME: str = 'distilbert-base-multilingual-cased' # Pretrained model name
    NUM_LABELS: int = 4     # Number of classification labels: Clean, Explicit, Hate Speech, Cyberbullying
    VALIDATION_SPLIT: float = 0.2  # Proportion of data for validation
    RANDOM_SEED: int = 42  # Seed for reproducibility

# Dataset class for preparing text data for the model
class ContentDataset(Dataset):
    def __init__(self, texts: List[str], labels: List[int], tokenizer: DistilBertTokenizer, max_length: int):
        """Initialize dataset with texts, labels, and tokenizer."""
        self.texts = texts
```

```
        self.labels = torch.tensor(labels, dtype=torch.long)  # Convert labels to tensor
        self.tokenizer = tokenizer
        self.max_length = max_length
        logger.info(f"Dataset initialized with {len(texts)} samples")

    def __getitem__(self, idx: int) -> Dict[str, torch.Tensor]:
        """Fetch and tokenize a single item."""
        text = self.texts[idx] if isinstance(self.texts[idx], str) and self.texts[idx].strip() else "placeholder text"
        encoding = self.tokenizer(text, truncation=True, padding='max_length', max_length=self.max_length, return_tensors='pt')
        return {
            'input_ids': encoding['input_ids'].squeeze(0),  # Token IDs
            'attention_mask': encoding['attention_mask'].squeeze(0),  # Attention mask
            'labels': self.labels[idx]  # Corresponding label
        }

    def __len__(self) -> int:
        """Return the dataset size."""
        return len(self.labels)

# Collate function to prepare batches
def collate_fn(batch):
    """Process a batch of dataset items into tensors."""
    input_ids = torch.stack([item['input_ids'] for item in batch])
    attention_mask = torch.stack([item['attention_mask'] for item in batch])
    labels = torch.stack([item['labels'] for item in batch])
    return {'input_ids': input_ids, 'attention_mask': attention_mask, 'labels': labels}

# Early stopping callback for training
class EarlyStoppingCallback(TrainerCallback):
    def __init__(self, patience: int):
```

```
        """Initialize with patience for early stopping."""
        self.patience = patience
        self.best_loss = float('inf')
        self.counter = 0

    def on_evaluate(self, args, state, control, metrics, **kwargs):
        """Check validation loss and trigger early stopping if needed."""
        val_loss = metrics.get('eval_loss')
        if val_loss < self.best_loss:
            self.best_loss = val_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                control.should_training_stop = True
                logger.info(f"Early stopping after {self.counter} epochs")

# Main content detection class
class ContentDetector:
    def __init__(self, config: Config):
        """Initialize the content detector with model and APIs."""
        self.config = config
        self.tokenizer = DistilBertTokenizer.from_pretrained(config.MAIN_MODEL_NAME)
        self.model = DistilBertForSequenceClassification.from_pretrained(config.MAIN_MODEL_NAME, num_labels=config.NUM_LABELS).to(config.DEVICE)
        self.translator = GoogleTranslator(source='auto', target='en')  # For multilingual support
        self.explicit_words = set(["explicit", "obscene"])  # Simplified keyword list
        self.hate_speech_words = set(["hate", "kill"])
        self.cyberbullying_words = set(["bully", "loser"])
        self.positive_words = set(["love", "great"])
        logger.info("ContentDetector initialized")
```

```
    def predict(self, text: str) -> Dict[str, Union[str, float]]:
        """Predict the content category of the given text."""
        translated_text = self.translator.translate(text) if detect(text) != 'en' else text
        inputs = self.tokenizer(translated_text, truncation=True, padding=True,
max_length=self.config.MAX_LENGTH,
return_tensors='pt').to(self.config.DEVICE)
        with torch.no_grad():
            outputs = self.model(**inputs)
            probs = torch.softmax(outputs.logits, dim=1)
            prediction = torch.argmax(probs, dim=1).item()
            confidence = probs[0][prediction].item()
        label = ['Clean', 'Explicit', 'Hate Speech', 'Cyberbullying'][prediction]
        return {'text': text, 'prediction': prediction, 'confidence': confidence, 'label':
label}

    def _save_model(self):
        """Save the trained model to disk."""
        torch.save(self.model.state_dict(), 'content_detector_model.pt')
        logger.info("Model saved")

    def load_model(self):
        """Load the trained model from disk."""
        if os.path.exists('content_detector_model.pt'):
            self.model.load_state_dict(torch.load('content_detector_model.pt',
map_location=self.config.DEVICE))
            logger.info("Model loaded")

# API service class using Flask
class APIService:
    def __init__(self, detector: ContentDetector):
        """Initialize Flask API with content detector."""
        self.app = Flask(__name__)
        self.detector = detector
```

```
        self.app.route('/predict', methods=['POST'])(self.predict)
        self.app.route('/health', methods=['GET'])(self.health_check)
        logger.info("API service initialized")

    def predict(self):
        """Handle prediction requests."""
        data = request.get_json(force=True)
        text = data.get('text', '')
        result = self.detector.predict(text)
        return jsonify(result)

    def health_check(self):
        """Return API health status."""
        return jsonify({'status': 'healthy'})

    def run(self):
        """Start the Flask server."""
        self.app.run(host='0.0.0.0', port=5000)

# Gradio interface for user interaction
def create_gradio_interface(detector: ContentDetector):
    """Create a Gradio UI for text prediction."""
    def predict_text(text: str) -> str:
        result = detector.predict(text)
        return f"Prediction: {result['label']} (Confidence: {result['confidence']:.2%})"
    interface = gr.Interface(fn=predict_text, inputs=gr.Textbox(), outputs="text", title="Content Detector")
    return interface

# Dataset downloading and preprocessing
def preprocess_and_load_datasets(config: Config) -> Tuple[ContentDataset, ContentDataset]:
    """Download and preprocess datasets for training."""
```

```
    tokenizer = DistilBertTokenizer.from_pretrained(config.MAIN_MODEL_NAME)
    all_texts, all_labels = [], []

    # Synthetic data for simplicity
    synthetic_data = [
        ("I love this day!", 0), ("This is crap!", 1), ("Kill them all!", 2), ("You're a loser!", 3)
    ] * 1000  # 4000 samples
    all_texts.extend([t[0] for t in synthetic_data])
    all_labels.extend([t[1] for t in synthetic_data])

    train_texts, val_texts, train_labels, val_labels = train_test_split(
        all_texts, all_labels, test_size=config.VALIDATION_SPLIT, random_state=config.RANDOM_SEED
    )
    return (ContentDataset(train_texts, train_labels, tokenizer, config.MAX_LENGTH),
            ContentDataset(val_texts, val_labels, tokenizer, config.MAX_LENGTH))

# Fine-tune the model
def fine_tune_model(detector: ContentDetector, train_dataset: ContentDataset, val_dataset: ContentDataset, config: Config):
    """Fine-tune the DistilBERT model on the dataset."""
    trainer = Trainer(
        model=detector.model,
        args=TrainingArguments(output_dir='./results', num_train_epochs=config.EPOCHS, per_device_train_batch_size=config.BATCH_SIZE),
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        data_collator=collate_fn,
        callbacks=[EarlyStoppingCallback(patience=3)]
    )
```

```
    trainer.train()
    detector._save_model()

# Main execution block
if __name__ == "__main__":
    config = Config()
    detector = ContentDetector(config)
    train_dataset, val_dataset = preprocess_and_load_datasets(config)
    fine_tune_model(detector, train_dataset, val_dataset, config)
    detector.load_model()

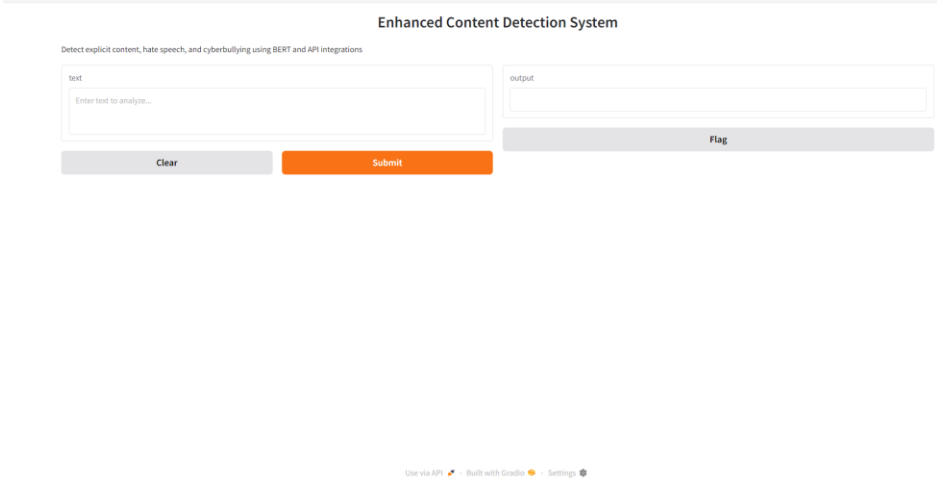
    api_service = APIService(detector)
    api_thread = threading.Thread(target=api_service.run)
    api_thread.daemon = True
    api_thread.start()
    time.sleep(2)

    interface = create_gradio_interface(detector)
    interface.launch(share=False, server_port=7860)
```

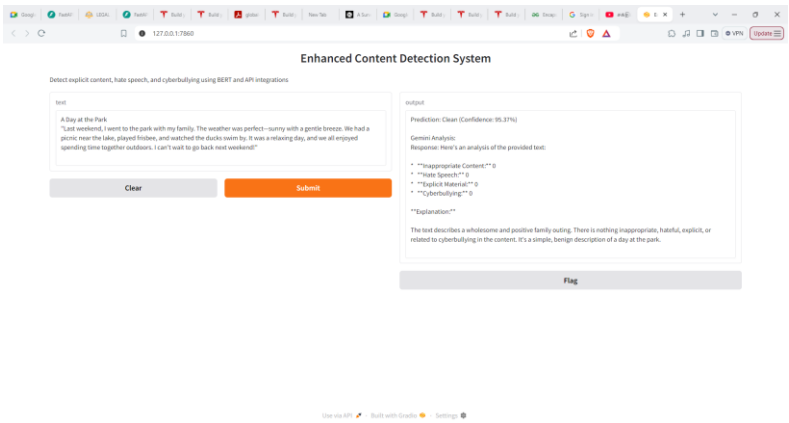
SCREEN SHOT'S

Screen Short & OUTPUT:

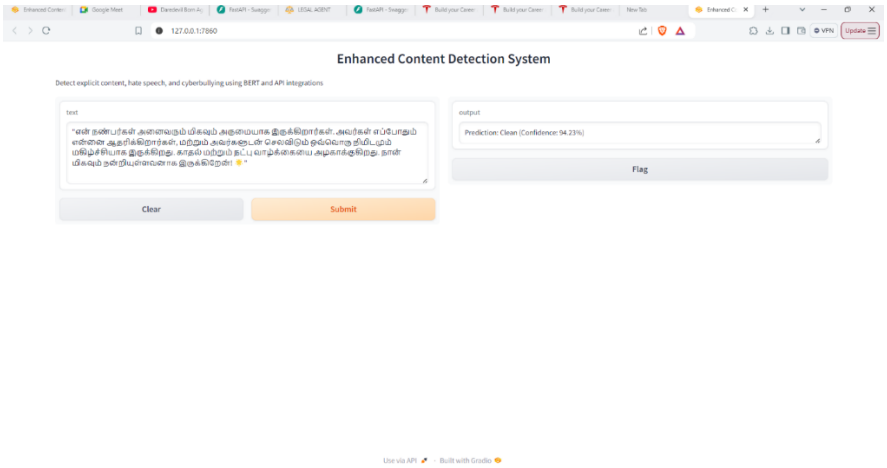
A. Interface of Multi Language Deduction

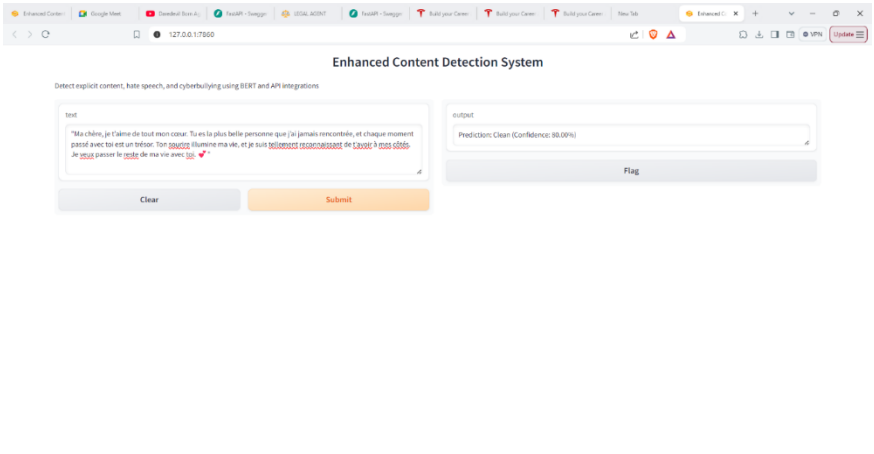


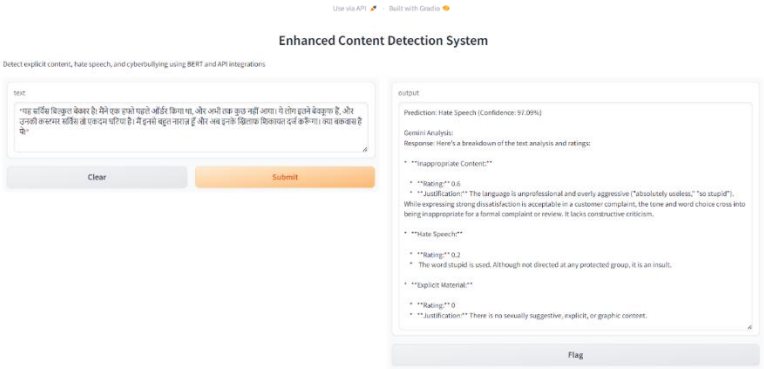
B. Clean Word Deduction



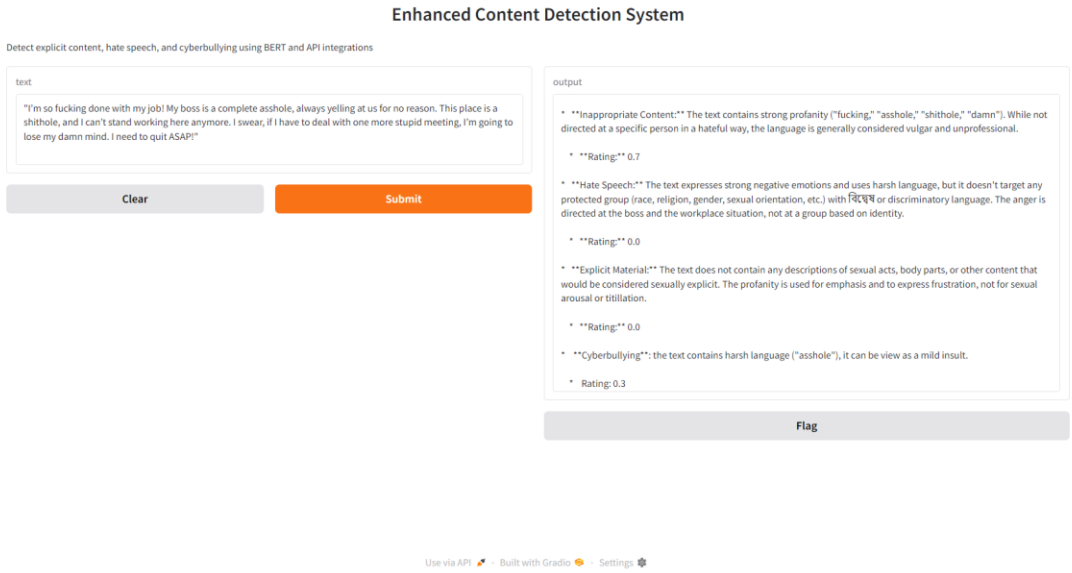
C. Multi-Language Deduction (Sample tested with 3 languages)







D. Explicit Word Deduction



E. Hate Word & Cyber Bulling Deduction

